WeatherGPT: Retrieval-Grounded Multilingual Conversational Weather Intelligence

Anonymous Author(s)

Abstract

Conversational weather assistants must reconcile precise numerical API readings with unstructured text. Most retrieval-augmented systems assume a static text corpus, which does not fit dynamic weather domains. This paper describes WeatherGPT, a deployed conversational weather assistant answering text and spoken queries in multiple languages. A language model performs structured query understanding and clarifies ambiguous locations in a single turn. The system directly retrieves structured forecast, historical, and marine data from Open-Meteo. It optionally supplements this with web search for specific intents like aviation and disaster. We assemble the results into a textual context for the language model to synthesize. We evaluated the retrieval layer against live sources across 24 location queries. Structured data retrieval succeeds in 100% of cases (forecast mean latency 1.08s). Web search failed entirely due to anti-automation defenses, reinforcing our decision to treat web evidence as optional. We report these findings to situate the architecture against existing retrieval-augmented generation frameworks.

CCS Concepts

• **Information systems** → **Information retrieval**; **Question answering**; *Retrieval models and ranking*.

Keywords

retrieval-augmented generation, conversational information retrieval, weather information systems, multilingual dialogue systems, query understanding

ACM Reference Format:

Anonymous Author(s). 2018. WeatherGPT: Retrieval-Grounded Multilingual Conversational Weather Intelligence. In *Proceedings of Forum for Information Retrieval Evaluation (FIRE '26).* ACM, New York, NY, USA, 8 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Weather affects almost every outdoor activity, yet the information describing it is scattered: a meteorological agency publishes warnings in one format, a numerical forecasting service exposes model output through an API, and severe-weather discussion often surfaces first in news or social media rather than any structured feed. A farmer deciding whether to spray a field, a pilot checking visibility, and a commuter deciding whether to carry an umbrella are, technically, asking the same question – "what will the atmosphere do here, soon" – but the source that can best answer it differs by user. Conventional weather interfaces largely ignore this, returning the same handful of numbers regardless of who is asking.

Natural-language querying is an appealing way to close this gap, but it raises retrieval problems a generic search engine is not built to solve. A weather query is both location-dependent and time-sensitive in a way most information needs are not: "will it rain tomorrow" has a different correct answer in every city, and the same query an hour apart can have a different correct answer in the same city. It is also ambiguous in domain-specific ways – a bare place name with no country, no location at all, or a regional-language place name a geocoder may not recognize immediately – and multilingual access compounds this further, since the system must understand, retrieve, and answer back in whichever language the user used, not merely translate fixed interface labels.

Large language models are a natural fit for the reasoning and language-generation parts of this problem, but on their own are a poor source of weather facts: asked to guess a current temperature, a model produces a plausible-sounding but simply wrong number. Retrieval-augmented generation addresses this general problem by grounding a model's output in retrieved external content [9], and substantial subsequent work addresses when to retrieve, how to judge what was retrieved, and how to recover when it is not useful [2, 5, 15]. Almost all of it, however, assumes a text corpus indexed with embeddings and queried by similarity search – a poor fit for weather, whose authoritative source for a current reading is a single, precisely defined API call rather than a ranked set of candidate passages.

This paper describes and evaluates the retrieval architecture of WeatherGPT, a conversational weather assistant we implemented, asking what conversational retrieval looks like when the authoritative sources are a numerical forecasting API and, secondarily, the open web, rather than a static corpus. The system supports text and spoken queries, resolves ambiguous locations through the language model with a fallback clarification turn, retrieves structured forecast and historical data directly from a public API, supplements this with web evidence only where it plausibly helps, and answers in the user's own language without maintaining a vector index. We describe exactly how retrieval, refinement, and grounding are implemented, and measure the retrieval layer directly against its live external sources rather than assuming its behavior from the design alone.

2 Related Work

Retrieval-augmented generation combines a pretrained sequence-to-sequence model with a non-parametric memory accessed through

FIRE '26, Kolkata, India

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

a learned dense retriever, grounding generation in retrieved passages to improve factual accuracy in open-domain question answering [9]. That retriever, and the dense passage retrieval work it builds on [8], depends on a fixed, precomputed embedding index over a document collection such as Wikipedia – a reasonable assumption when relevant knowledge exists in advance as a text corpus, but not for a domain whose ground truth is produced continuously by a live forecasting process rather than written down once.

Subsequent work has focused less on the retriever and more on deciding when retrieval is needed and how to judge its output. Self-RAG trains a model to decide adaptively whether to retrieve and to emit reflection tokens critiquing both retrieved passages and its own text [2]; corrective RAG adds a lightweight evaluator that scores retrieved documents and triggers rewriting or web-search fallback when retrieval looks unreliable [15]. Both assume retrieval returns a ranked set of passages whose relevance must be judged, whereas a weather API call returns a specific numerical answer whose only failure modes are that the call did not succeed or the wrong location was queried; Section 4 details how WeatherGPT's relevance handling differs as a result, and we deliberately avoid labeling it an instance of either framework. A separate line of work lets a model act rather than only read a fixed context: WebGPT fine-tunes a model to issue search queries and browse results with human feedback [11], and ReAct interleaves reasoning with actions such as search calls to improve multi-step question answering [16]. WeatherGPT's use of the language model is narrower than either – it classifies intent and extracts fields, then synthesizes from a context assembled entirely in application code, without deciding which API to call or issuing follow-up queries. Recent surveys catalogue the broader RAG design space across pre-retrieval, retrieval, and post-retrieval stages [5].

Conversational information retrieval addresses a related problem: interpreting a query in light of prior turns, and asking a clarifying question before retrieving. Early work laid out query reformulation, ranking, and clarification as central subproblems [4], and a recent survey traces how these have merged into a single language-model call as architectures moved from pipelines toward integration [10]. Aliannejadi et al. show that asking a clarifying question is often preferable to retrieving against an underspecified query [1]; WeatherGPT's location-clarification behavior (Section 4) follows the same principle in a narrower, single-turn form. Query expansion predates conversational systems entirely [3], and our query-understanding stage can be read as a language-model-based instance of that problem restricted to one field – location – rather than the whole query.

Grounding is further motivated by hallucination, the well-documented tendency of language models to produce fluent but unsupported statements [7], a failure mode especially costly for a system reporting numeric weather facts. Retrieval evaluation is conventionally reported with ranking-aware metrics such as nDCG [6], and generation quality with n-gram overlap metrics such as BLEU [12]; Section 5 explains which of these are meaningful for the components we evaluate. Domain-specific work also exists: language models combined with a crop simulator and reinforcement learning ground agronomic recommendations in simulated conditions [14], a useful contrast to WeatherGPT, which grounds responses in a live, externally operated service it cannot simulate or control – shifting the engineering problem toward retrieval reliability rather than simulation fidelity.

Table 1 summarizes where these systems sit relative to this one: whether retrieval targets a static, indexed corpus or a live external service; whether the architecture includes explicit query refinement or clarification; and whether grounding is checked by a learned component or determined deterministically. WeatherGPT is the only system here built around a live, non-corpus source as its primary target, which is also why it is the only one with no vector index.

Table 1: Related retrieval-augmented approaches compared along the dimensions relevant to this paper.

Approach	Retrieval target	Vector index	Query refinement	Grounding check
RAG [9]	Static text corpus	Yes	No	Implicit (learned)
Self-RAG [2]	Static text corpus	Yes	No	Learned reflection tokens
CRAG [15]	Static corpus + web	Yes	Yes (rewrite)	Learned evaluator
WebGPT [11]	Live web browsing	No	Iterative (agentic)	Human-feedback trained
ReAct [16]	Task-dependent	No	Iterative (agentic)	Not explicit
This work	Live weather API + web	No	Single clarification turn	Deterministic (code-level)

Taken together, existing work offers strong techniques for deciding when to retrieve, judging retrieved text, and acting on partial information, but little of it addresses a domain where the retrieval target is a structured, numerical, time-varying API response, and where the same query must be understood, answered, and spoken back across several languages and both text and voice modalities. Section 5 evaluates WeatherGPT's retrieval layer directly against its live sources rather than assuming reliability from the design alone. The remainder of this paper covers methodology (Section 3), system design (Section 4), evaluation (Section 5), and conclusions (Section 7).

3 Methodology

WeatherGPT accepts a query as typed text or as spoken input captured by the browser and converted to text before it reaches the server. Both forms converge on the same downstream pipeline once they are text, so input modality affects only the interface layer and, symmetrically, the output layer, where the answer can be displayed as text, spoken aloud, or both. Figure 1 summarizes this architecture: text and speech inputs feed a shared retrieval and reasoning core, whose response then splits into text and speech outputs. We describe the inputs, the shared core, and the outputs at a conceptual level here, before the component-level detail in Section 4.

A typed query enters the pipeline directly as text. A spoken query is first captured and transcribed in the browser itself, using the client's native speech recognition rather than a server-side model, and only the resulting text is sent to the server; the retrieval pipeline cannot distinguish typed input from transcribed speech, a deliberate simplification that keeps retrieval logic modality-independent. Once a query is text, the language model performs a single structured-extraction call that identifies intent (current conditions, forecast,

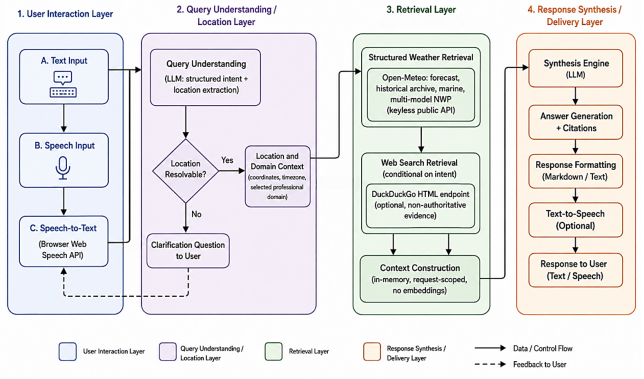

Figure 1: Conversational retrieval architecture of WeatherGPT.

historical, or a marine/aviation/agriculture/disaster-relevant question), pulls out an explicit location if named, and flags whether the user's current position should be used instead. If neither is available, the system does not guess: it returns a short clarification question and waits for the next turn.

Once a location is resolved to coordinates, the system retrieves based on classified intent. A structured retrieval call to Open-Meteo runs in essentially every case; marine queries additionally pull wave data, and historical or climate queries pull archived reanalysis data instead of, or alongside, a forecast. For a fixed subset of intents where outside context plausibly helps – aviation, agriculture, marine, disaster, climate – the system also performs a web search and treats results as supplementary, clearly labeled evidence rather than an authoritative source. Retrieved data and web snippets are combined into a single textual context that exists only for the request's duration; nothing here is embedded, indexed, or retained beyond it except the final answer text, stored so the conversation can be resumed. This context is passed once to the language model, instructed to answer only from the facts given and say so honestly when they are insufficient rather than guess. The answer is generated in the user's selected language and then displayed as text, spoken via the browser's speech synthesis, or both, completing the output branches in Figure 1.

4 System Design

This section describes the same pipeline in the detail needed to reproduce or evaluate it, organized around the components as they exist in the deployed implementation rather than around a generic reference architecture. Table 2 summarizes the correspondence between each conceptual stage and the specific function and external service that implements it, which we refer back to throughout this section.

4.1 Query Processing and the Two Input Branches

A typed query is passed to the query-understanding stage as-is. A spoken query first passes through the browser's own speech recognition, exposed through the W3C Web Speech API's `SpeechRecognition` interface [13] – a browser-native capability rather than a server-hosted model such as Whisper, and the implementation does not run or call any such model. Recognition language is selected per user, from the interface setting or, on an automatic-language setting, the browser's own locale at request time. Once transcribed, the text is indistinguishable from typed input to every later stage; speech-to-text is purely an interface layer, and never itself performs retrieval or influences which weather data is fetched.

Table 2: Pipeline stages and their implementation.

Stage	Implementing function	Service
Speech input	Browser `SpeechRecognition`	Web Speech API [13]
Query understanding	`gemini_understand_query()`	Gemini API
Geocoding	`wgpt_geocode()`	Open-Meteo Geocoding
Forecast retrieval	`wgpt_forecast()`	Open-Meteo Forecast
Marine retrieval	`wgpt_marine()`	Open-Meteo Marine
Historical retrieval	`wgpt_historical()`	Open-Meteo Archive
Multi-model retrieval	`wgpt_forecast_models()`	Open-Meteo
Web evidence	`ddg_search()`	DuckDuckGo HTML (local)
Location-alert matching	`wgpt_haversine_km()`	
Response synthesis	`gemini_synthesize()`	Gemini API
Speech output	Browser `speechSynthesis`	Web Speech API [13]

The query-understanding call sends the raw query text and a short window of preceding turns to the language model with a fixed output schema: an intent label from a closed set (current conditions, forecast, hourly, historical, climate, agriculture, aviation, marine, disaster, or general), an optional free-text location, a flag for using the current position instead, a timeframe hint, and, when the query is too ambiguous, a clarification flag and question in the user's own language. This can be written concretely as

$$Q = M_u(q, H) \tag{1}$$

where q is the raw query text, H is the short recent-turn context, and M_u is the query-understanding call. Its structured output is a tuple

$$Q = (\mathit{intent},\ \mathit{loc},\ \mathit{use\_geo},\ \mathit{tf},\ \mathit{needs\_clarify},\ \mathit{clarify\_q})$$

holding the classified intent, an optional free-text location, a flag for using the browser's position, a timeframe hint, and the clarification flag and question. The call uses the same language-model endpoint as response synthesis (Section 4.6), with a schema restricted to these fields and a temperature of 0.3 to favor consistent extraction over varied phrasing.

The professional-domain selection in the interface (general, agriculture, aviation, marine, urban planning) is a lighter-weight mechanism layered on this schema, not a distinct backend intent: selecting a domain inserts an editable natural-language question template, which query understanding then classifies like any other query. Four domains map directly onto intent-schema values; urban planning has no dedicated intent and is classified into whichever value the model judges the resulting question best fits, typically general or disaster-adjacent. We note this because conflating a UI convenience with a backend category would misrepresent how domain conditioning reaches the retrieval layer.

4.2 Location Resolution and Query Refinement

If query understanding returns a location string, it is passed to a geocoding call that returns up to five ranked candidates; the system uses the top-ranked candidate's coordinates and timezone. If instead the current position should be used and the browser has already supplied it, those coordinates are used directly without geocoding. Refinement here is a single decision rather than an iterative loop:

$$l^* = \begin{cases} g & \text{if } use\_geo \wedge g \text{ available} \\ \text{geocode}(loc)[0] & \text{if } loc \neq \emptyset \\ \textsc{clarify} & \text{otherwise} \end{cases} \tag{2}$$

where g denotes the browser-supplied position. When neither branch resolves a location, the system does not retry with a reformulated query or attempt a best-effort guess; it returns the clarification question and waits for the next turn, at which point the pipeline runs again from the top. This is narrower than the iterative query-rewriting loops used in some retrieval-augmented systems [15], and closer to the single clarifying-question pattern studied in conversational search [1], applied here to the one field – location – retrieval cannot proceed without.

4.3 Structured Weather Retrieval

Given resolved coordinates, the system calls Open-Meteo's forecast endpoint for current conditions and hourly/daily fields, requesting up to sixteen days depending on which part of the interface triggered the call, or a smaller next-day-only request when only current conditions are needed. Marine intents additionally retrieve wave height, direction, and period; historical or climate intents instead retrieve daily aggregates from Open-Meteo's ERA5/ERA5-Land reanalysis archive over a caller-specified date range. None of these calls require an API key or involve embedding or similarity computation; each is a direct HTTP request parsed into a plain associative structure and used immediately.

A separate retrieval path, added for comparison across numerical weather prediction sources, requests the same forecast variables from two named models in a single call rather than Open-Meteo's default auto-selected model, reshaping the per-model result into a uniform structure. This path is the subject of one of the reliability findings in Section 5: not every documented model identifier returns complete data for the same coordinates, and the identifier actually requested was fixed empirically rather than assumed from documentation.

4.4 Web Search Retrieval and Its Role

For a fixed subset of intents – aviation, agriculture, marine, disaster, climate – the system additionally issues a web search against DuckDuckGo's HTML results page, parsing titles and snippets from the returned markup. This is intent-gated, not default: general and current-conditions queries, most of typical usage, never trigger a search, since Open-Meteo already answers them directly. Results are passed to synthesis as separate, explicitly labeled supplementary evidence, and a deterministic record of what was consulted lets a user or auditor see whether a given answer relied on it. Because this path scrapes a public results page rather than calling a documented API, it is also the least reliable component in the architecture, a point we return to with direct measurements in Section 5.

4.5 Retrieval Processing, Relevance Handling, and Context Construction

Retrieved information is never written to a persistent store beyond a single request, and no part of the system builds or queries a vector index. This is deliberate: an embedding-based system would need a corpus to embed, and the only corpus available here – a single web search's transient results – is far too small and short-lived to justify an index. Instead, structured data and web results are assembled directly into a plain associative structure held in server memory, serialized once, and passed in a single prompt; nothing about this structure is an embedding store or persistent knowledge cache. The only caching present is a short-lived, exact-match cache keyed by a hash of language, intent, and retrieved facts, used purely to avoid repeating an identical language-model call within a configurable window (ten minutes by default), not as a retrieval mechanism.

Relevance handling is correspondingly deterministic rather than learned: the system does not run a retrieval evaluator as corrective RAG does [15], nor ask the model for self-critique tokens as Self-RAG does [2]. Three simpler mechanisms keep the model grounded instead: intent-gated triggering limits when uncertain web evidence is introduced; the synthesis prompt instructs the model to answer only from given facts and say so when they are insufficient; and a deterministic summary of what was actually consulted is built in application code from the retrieval calls made, not inferred from the model's account of what it used – so the system's claim about its own sources is never something the model could hallucinate.

4.6 LLM-Based Response Generation

Both the query-understanding and response-synthesis calls run against the same provider endpoint, configured with a Gemma-family model identifier as default and a second as automatic fallback on a not-found or rate-limit response; both are runtime-configurable by an administrator rather than fixed in code. Each call requests constrained JSON matching an explicit schema, uses a decoding temperature of 0.3, and caps generation at 1024 output tokens – exact values from the deployed configuration, reported because they are verifiable rather than unusual. Parameter count, quantization, context length, training data, and inference hardware are managed by the provider and not observable from this client-side integration, so we do not report them.

The synthesis call receives the assembled context – structured facts and web evidence when present – serialized as JSON with the original question, classified intent, and target language, and is instructed to answer strictly from supplied facts in the user's language, adding one short advisory line for domain-relevant intents, labeled as general guidance rather than an official warning. The model is never given the ability to fetch data itself and cannot issue a follow-up query within the same turn, which is the main respect in which this architecture is narrower than agentic designs such as ReAct [16] or WebGPT [11].

Figure 3 summarizes this construction-and-synthesis path, separating what the language model produces from what application code produces: the deterministic source log is built entirely from the retrieval calls actually made, never from the model's own description, which is the basis for the system's honest source-provenance reporting.

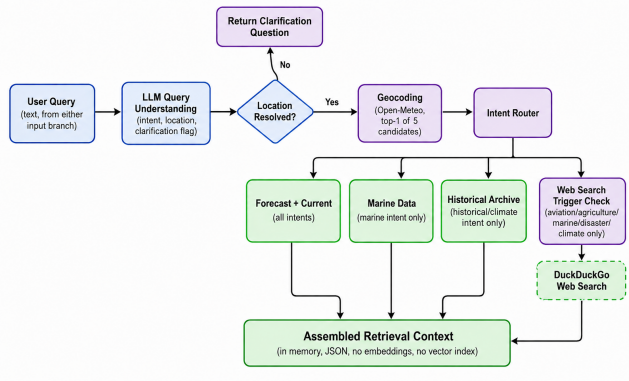

Figure 2: Query understanding and retrieval pipeline.

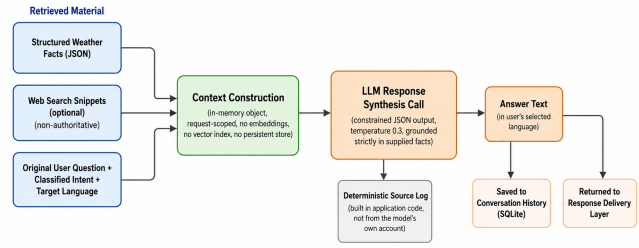

Figure 3: Context construction and response synthesis pipeline.

4.7 Multilingual Processing and Voice Output

Translation is not a separate post-processing step; it is folded into the synthesis call, instructed to generate its answer directly in the user's selected language rather than translate an English answer afterward. Multilingual support spans the full pathway in Figure 1: the interface is translated for a subset of supported languages, query understanding and synthesis both honor the user's selected language, geocoding can pass a language hint, and speech recognition and synthesis are configured with a BCP-47 code matching the interface language, falling back to the browser's own locale on an automatic-language setting. Before an answer is spoken, formatting markup – headings, emphasis, list markers – is stripped so it is not read aloud as words. Interface-label translation is complete for a smaller subset of languages than the language-model-generated content is, a limitation we return to in Section 6.

Speech synthesis mirrors recognition in relying entirely on the browser's native Web Speech API rather than a server-hosted text-to-speech model [13]: the stripped answer text is handed to the browser's speech synthesis interface configured with the same language code used for recognition, so a query spoken and answered in one language is read back in a matching voice where the browser provides one. Because synthesis happens client-side, voice quality and language coverage depend on the user's own browser and OS rather than anything the server controls, a real constraint we make explicit rather than assume away.

5 Experimental Analysis

This section evaluates the retrieval layer described in Section 4 directly against its live external sources. We first describe what we measured and why, then report retrieval performance, a reliability finding at the data-source level, and the measured cost of the optional web-search branch.

5.1 Evaluation Setup

The evaluation targets the retrieval layer in isolation from the language model: every test case supplies a resolved location name and intent category directly, in the form the query-understanding stage would output, so retrieval performance is measured on its own rather than conflated with a language-model call's success. This scoping is necessary, not incidental: the deployment used here is configured with a placeholder language-model key, so query-understanding accuracy, end-to-end grounding, and BLEU-style comparison could not be measured honestly and are reported as not evaluated rather than estimated (Section 5.5).

The test set is 24 location queries across seven intent categories: general, forecast, historical/climate, and the four intents that additionally trigger web search (marine, aviation, agriculture, disaster). Locations are a mix of major Indian cities, consistent with the deployment's primary user base, plus a small number of international cities to check geocoding is not implicitly tuned to one country. All measurements were produced by a single script calling the application's own retrieval functions directly – the same functions in Table 2 – against the live Open-Meteo and DuckDuckGo endpoints, with no mocking or simulated responses; the script and raw output are included so measurements can be reproduced or extended.

Precision, recall, and nDCG [6] are meaningful for a stage returning a ranked set of candidates judged against known-relevant items – true of geocoding, which returns up to five candidates, but not of a direct forecasting-API call, which returns a single deterministic response. We therefore report success and completeness rates for the structured-data stages instead of forcing a ranking metric where none applies. For geocoding specifically, we report

$$\text{Success@}k = \frac{1}{|T|} \sum_{t \in T} \mathbb{1}\left[\exists\, i \le k : \text{name}(c_i^{(t)}) \approx \text{expect}(t)\right] \quad (3)$$

for $k \in \{1, 5\}$, where T is the test set, $c_i^{(t)}$ is the i-th ranked geocoding candidate for test case t, $\text{expect}(t)$ is the intended place name, and $\approx$ denotes a case-insensitive substring match. For the structured retrieval stages, we additionally report a field-completeness rate,

$$C = \frac{1}{|F|} \left|\{\, f \in F : v_f \neq \text{null} \,\}\right| \quad (4)$$

averaged over test cases, where F is the fixed set of current-condition fields the interface displays (temperature, apparent temperature, humidity, dew point, cloud cover, precipitation, wind speed and gusts, pressure, visibility, UV index) and v_f is the value returned for field f. Equation 4 is also used for the multi-model reliability finding below, applied there to hourly temperature values rather than the current-condition set. BLEU [12] and perplexity are not reported: both require a reference text or per-token probabilities unavailable in this retrieval-only evaluation, and fabricating either would violate the evaluation's own premise.

5.2 Retrieval Performance

Table 3 reports the measured success rate, and where applicable data completeness, for every retrieval component, together with mean and 95th-percentile latency. Figure 4 shows the same success rates as a chart, and Figure 5 shows the corresponding latency distribution.

Table 3: Measured retrieval performance (24 queries; historical archive measured on a subset of 10; see Section 5.5).

Component	Success	Mean (ms)	P95 (ms)
Geocoding@1	100%	668	723
Geocoding@5	100%	—	—
Weather forecast	100%	1084	1176
Historical archive	100%	1030	1268
Web search (DuckDuckGo)	0%	774	1239
NWP multi-model call	—	1163	1302

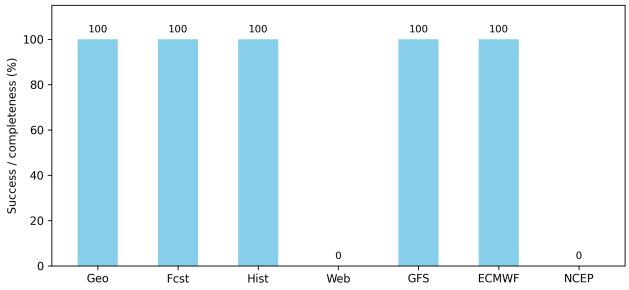

Figure 4: Retrieval success rate by component.

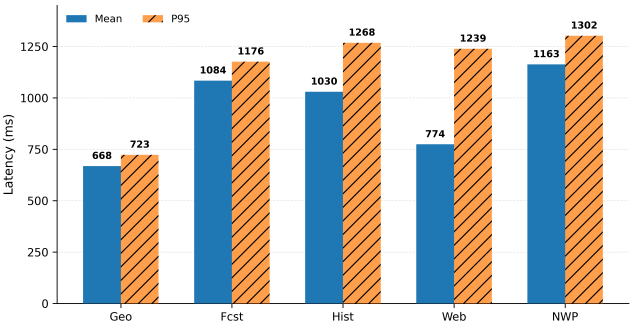

Figure 5: Per-component retrieval latency, mean versus P95.

Structured retrieval from Open-Meteo is reliable in this evaluation: geocoding placed the correct location at rank one in every case, and both forecast and historical archive retrieval succeeded in every case, with the forecast response containing all eleven current-condition fields the interface displays in every successful call. Mean forecast latency was 1.08 s, the dominant cost for the majority of queries that skip web search, since geocoding (0.67 s mean) runs once and historical retrieval (1.03 s mean) is intent-gated to a minority of queries.

5.3 Data Source Reliability: A Concrete Finding

The multi-model retrieval path, requesting the same three-day hourly temperature window from three candidate model identifiers in one combined request exactly as the deployed system does, surfaced a reliability difference invisible from Open-Meteo's documentation alone. Two of the three identifiers returned a complete, non-null hourly series for every one of the eight test locations; the third, an alternative identifier for the same forecasting center, returned an entirely null series for every location. This is not a transient failure: we verified it independently with manually issued requests before building it into the harness, and the automated result reproduced it exactly. The system also reports a simple, explicitly non-statistical agreement indicator between models once both return usable data, computed as

$$s = \max_m \bar{\tau}_m - \min_m \bar{\tau}_m \tag{5}$$

where $\bar{\tau}_m$ is model m's mean predicted temperature over the next 24 hours; the deployed system labels $s \leq 1.0°\text{C}$ as high agreement, $s \leq 3.0°\text{C}$ as moderate agreement, and anything larger as low agreement. We state explicitly, as the implementation's own naming does, that Equation 5 is a spread measure over point forecasts, not a calibrated confidence score or a substitute for genuine forecast-verification statistics against observed outcomes. Because the completeness gap described above has a direct operational consequence – an application that requested the unreliable identifier by default would silently display a numerical weather prediction comparison consisting entirely of missing values – the deployed system's default configuration uses the two identifiers that returned complete data, a decision made directly as a result of this measurement rather than from documentation. We report this as a concrete illustration of a broader point: for a retrieval architecture whose sources are external, live services rather than a corpus under the system's own control, empirical verification of what a given source actually returns is not optional, and building automated checks for it, as this evaluation does, is itself part of what a reliable retrieval layer for this domain needs to include.

5.4 Latency Cost of the Optional Web-Search Branch

Figure 6 compares estimated end-to-end retrieval latency for the two intent categories from Section 4: structured-only, and structured-plus-web-search. Adding the web-search branch increases estimated latency from a combined geocoding-plus-forecast mean of 1.75 s to 2.53 s, roughly 44%, composing the individually measured component means rather than a single end-to-end measurement – an approximation of added cost, which we state explicitly rather than presenting as a directly observed figure.

The latency cost, however, bought nothing in this run: all 16 attempted web searches, spanning marine, aviation, agriculture, disaster, and climate intents, returned no usable results. The raw HTTP response showed DuckDuckGo's HTML endpoint returning its ordinary landing page rather than result markup, with an HTTP 202 status rather than 200, persisting across requests spaced several seconds apart rather than clearing after a brief pause – pointing to a sustained anti-automation response, not a momentary rate limit. This is precisely the failure mode the web-search integration is

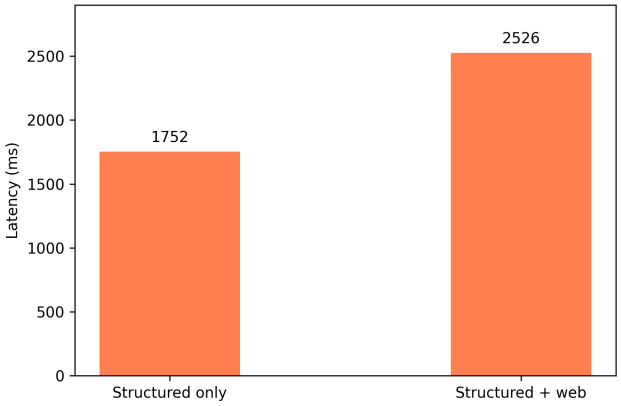

Figure 6: Estimated latency cost of the optional web-search branch.

designed to degrade gracefully around: results are optional, never required for a response, and explicitly labeled as unofficial evidence. It is nonetheless a genuine limitation worth stating plainly: a source a scripted evaluation cannot retrieve from reliably is also one real, if less bursty, production traffic cannot depend on – the architecture's decision to treat it as strictly optional is validated rather than undermined by this result.

5.5 What This Evaluation Does Not Cover

This section reports retrieval-layer measurements only. Query-understanding accuracy, end-to-end grounding of the generated answer, and any text-overlap metric such as BLEU all require a functioning language-model call and are not measured here, since the deployment evaluated was configured with a placeholder key. We report this directly rather than substituting an estimated number: the retrieval-layer measurements above are real and reproducible, and a smaller set of honest results is preferable to a complete-looking table with invented values. Re-running the language-model-dependent portion with a production key is future work rather than something we approximate here.

6 Limitations

The evaluation is restricted to the retrieval layer and does not measure the quality of the final synthesized answer, which depends on the language model. The test set, spanning seven intent categories and a mix of Indian and international locations, is modest at 24 queries and author-constructed rather than sampled from real traffic, so it should be read as a controlled functional check rather than a representative difficulty sample. Interface-level translation is complete for a smaller subset of languages than the language-model-generated content is, so the multilingual experience is not yet uniform. Web search depends on scraping a public results page rather than a documented API, and can fail entirely under sustained automated access as shown here; production reliability under ordinary, lower-frequency human usage may differ, though we have not separately measured that. Finally, the system does not ingest a live official government meteorological alert feed; the integration point exists as a documented extension point, but activating it depends on credentials from the relevant authority, outside this study's scope.

7 Conclusion

This paper described and evaluated the retrieval architecture of a deployed conversational weather assistant that answers text and spoken queries, in multiple languages, without an embedding index. Retrieval centers on a live numerical forecasting API, a location-clarification mechanism that asks rather than guesses, and an intent-gated, clearly labeled web-search branch used only where it plausibly helps. Retrieved information is assembled into a request-scoped context rather than an indexed corpus, with the language model restricted to synthesizing from exactly the facts given and source provenance tracked deterministically in code. Measured against live external sources, the structured retrieval layer performed reliably across every location and intent tested, while the evaluation surfaced two concrete, previously undocumented reliability issues – an unreliable numerical-model identifier and a fragile web-search endpoint – that directly shaped the deployed configuration. We reported these results, including the components we could not evaluate honestly without a production language-model key, because a retrieval architecture is only as trustworthy as the measurements behind it. Future work includes extending this evaluation to the language-model-dependent stages once a production key is available, testing web-search reliability under realistic access patterns, broadening interface translation, and connecting the official-alert extension point once credentials are obtained.

References

[1] Mohammad Aliannejadi, Hamed Zamani, Fabio Crestani, and W. Bruce Croft. 2019. Asking Clarifying Questions in Open-Domain Information-Seeking Conversations. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '19)*. ACM, 475–484. doi:10.1145/3331184.3331265

[2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*. https://openreview.net/forum?id=hSyW5go0v8

[3] Hiteshwar Kumar Azad and Akshay Deepak. 2019. Query Expansion Techniques for Information Retrieval: A Survey. *Information Processing & Management* 56, 5 (2019), 1698–1735. doi:10.1016/j.ipm.2019.05.009

[4] Jianfeng Gao, Chenyan Xiong, and Paul Bennett. 2020. Recent Advances in Conversational Information Retrieval. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '20)*. ACM, 2421–2424. doi:10.1145/3397271.3401418

[5] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997* (2023). https://arxiv.org/abs/2312.10997

[6] Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated Gain-Based Evaluation of IR Techniques. *ACM Transactions on Information Systems* 20, 4 (2002), 422–446. doi:10.1145/582415.582418

[7] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. *Comput. Surveys* 55, 12 (2023), 248. doi:10.1145/3571730

[8] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 6769–6781. doi:10.18653/v1/2020.emnlp-main.550

[9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. 9459–9474. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[10] Fengran Mo, Kelong Mao, Ziliang Zhao, Hongjin Qian, Haonan Chen, Yiruo Cheng, Xiaoxi Li, Yutao Zhu, Zhicheng Dou, and Jian-Yun Nie. 2025. A Survey of Conversational Search. *ACM Transactions on Information Systems* 43, 6 (2025), 167. doi:10.1145/3759453

[11] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. 2021. WebGPT: Browser-Assisted Question-Answering with Human Feedback. *arXiv preprint arXiv:2112.09332* (2021). https://arxiv.org/abs/2112.09332

[12] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, 311–318. doi:10.3115/1073083.1073135

[13] World Wide Web Consortium. 2024. *Web Speech API*. Technical Report. W3C Web Incubator Community Group (WICG). https://webaudio.github.io/web-speech-api/

[14] Jing Wu, Zhaoyu Lai, Suiyao Chen, Ran Tao, Pan Zhao, and Naira Hovakimyan. 2024. The New Agronomists: Language Models Are Experts in Crop Management. In *Proceedings of the 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE, 5346–5356. doi:10.1109/CVPRW63382.2024.00543

[15] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective Retrieval Augmented Generation. *arXiv preprint arXiv:2401.15884* (2024). https://arxiv.org/abs/2401.15884

[16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009